

```
In [48]: #Write a program that uses class to store the marks of students.
#use list to store the marks in three subjects.
#also give total of marks

class Student:
    def __init__(self,name,roll_no):
        self.marks=[]
        self.name=name
        self.roll_no=roll_no
    def entermarks(self):
        for i in range(3):
            # m=int(input("Enter marks of %s in subject %d:"%(self.name,i+1))
            m=int(input("Enter marks of {} in subject {}: ".format(self.name,i+1)))
            self.marks.append(m)
    def total(self):
        return self.marks[0]+self.marks[1]+self.marks[2]
    def display(self):
        print("name=",self.name , ", roll number=", self.roll_no ,
              "got marks in three subjects" ,self.marks ,"\nTotal marks= ",self.total())
```

```
In [49]: s1=Student("pratham",123)
s1.entermarks()
s1.display()

Enter marks of pratham in subject 1: 80
Enter marks of pratham in subject 2: 90
Enter marks of pratham in subject 3: 70
name= pratham , roll number= 123 got marks in three subjects [80, 90, 70]
Total marks= 240
```

```
In [42]: #write a program with class employee that keeps record of no. of employees i
#also store their data.

class Employee:
    empcount=0
    def __init__(self,name,age,salary):
        self.name=name
        self.age=age
        self.salary=salary
        Employee.empcount+=1
    def display_count_emp(self):
        print("There are",Employee.empcount,"employees")
    def display(self):
        print("Employee name is",self.name," , age is",self.age,"and salary is",self.salary)
```

```
In [43]: emp1=Employee("s1",34,40000)

emp2=Employee("h1",36,50000)
emp2.display_count_emp()
print("Detail of employee-2")
emp2.display()

There are 2 employees
Detail of employee-2
Employee name is h1 , age is 36 and salary is 50000
```

In [1]: *#write a python program for library book record with oops.*

```
class library:
    def __init__(self):
        self.title=""
        self.author=""
        self.publisher=""
    def read(self):
        self.title=input("Enter Book Title: ")
        self.author=input("Enter Book author: ")
        self.publisher=input("Enter Book Publisher: ")
    def display(self):
        print("Title:", self.title)
        print("Author:", self.author)
        print("Publisher:", self.publisher)
        print("\n")
my_book=[]
ch='y'
while(ch=='y'):
    print('''
1. Add New Book
2. Display Books
''')
    choice=int(input("Enter choice: "))
    if(choice==1):
        book=library()
        book.read()
        my_book.append(book)
    elif(choice==2):
        for i in my_book:
            i.display()
    else:
        print("Invalid choice!")
    ch=input("Do you want to continue..?")
print("Bye!")
```

1. Add New Book
2. Display Books

Enter choice: 1
Enter Book Title: python
Enter Book author: rao
Enter Book Publisher: tata
Do you want to continue..?y

1. Add New Book
2. Display Books

Enter choice: 2
Title: python
Author: rao
Publisher: tata

Do you want to continue..?y

1. Add New Book
2. Display Books

Enter choice: 1
Enter Book Title: py1
Enter Book author: abc
Enter Book Publisher: tata
Do you want to continue..?y

1. Add New Book
2. Display Books

Enter choice: 1
Enter Book Title: py2
Enter Book author: abq
Enter Book Publisher: tata
Do you want to continue..?y

1. Add New Book
2. Display Books

Enter choice: 2
Title: python
Author: rao
Publisher: tata

Title: py1
Author: abc
Publisher: tata

Title: py2
Author: abq
Publisher: tata

Do you want to continue..?n
Bye!

```

In [ ]: #write a python program that has class store which keeps record of code and
#each product. Display a menu of all products to the user and prompt
#him to enter the quantity of each item required . generate a bill and displ
class Store:
    def __init__(self,n):
        self.item_name=[]
        self.item_code=[]
        self.price=[]
        self.n=n
    def get_data(self):
        #n=int(input("enter no of items: "))
        for i in range(self.n):
            self.item_name.append(input("enter name of item: "))
            self.item_code.append(int(input("enter code of item: ")))
            self.price.append(int(input("enter cost of item: ")))
    def display_data(self):
        print("Item Name \t Item Code \t Price")
        for i in range(self.n):
            print(self.item_name[i],"\t\t",self.item_code[i],"\t\t",self.p

    def calculate_bill(self,quant):
        total_amount=0
        for i in range(self.n):
            total_amount+=(self.price[i]*quant[i])
        print("*****Bill*****")
        print("ITEM NAME\t ITEM CODE\t PRICE \t QUANTITY \t SUBTOTAL")
        for i in range(self.n):
            print(self.item_name[i],"\t\t",self.item_code[i],"\t\t",
                self.price[i],"\t",quant[i],"\t\t",
                self.price[i]*quant[i])
        print("*****")
        print("Total= ",total_amount)

n=int(input("enter no of items user want to buy: "))
s1=Store(n)
s1.get_data()
s1.display_data()
q=[]
print("Enter quantity of each item: ")
for i in range(n):
    q.append(int(input("Enter quantity of item {} : ".format(i+1))))

s1.calculate_bill(q)

```

enter no of items user want to buy: 2

```
In [1]: """ Bank Account class:
Create a Python class called BankAccount which represents a bank account,
having as attributes: accountNumber (numeric type), name
(name of the account owner as string type), balance.
Create a constructor with parameters: accountNumber, name, balance.
Create a Deposit() method which manages the deposit actions.
Create a Withdrawal() method which manages withdrawals actions.
Create an bankFees() method to apply the bank fees with a
percentage of 5% of the balance account.
Create a display() method to display account details.
Give the complete code for the BankAccount class.
"""

class BankAccount:
    # create the constructor with parameters: accountNumber, name and balance
    def __init__(self, accountNumber, name, balance):
        self.accountNumber = accountNumber
        self.name = name
        self.balance = balance

    # create Deposit() method
    def Deposit(self, d):
        self.balance = self.balance + d

    # create Withdrawal method
    def Withdrawal(self, w):
        if(self.balance < w):
            print("impossible operation! Insufficient balance !")
        else:
            self.balance = self.balance - w

    # create bankFees() method
    def bankFees(self):
        self.balance = (95/100)*self.balance

    # create display() method
    def display(self):
        print("Account Number : ", self.accountNumber)
        print("Account Name : ", self.name)
        print("Account Balance : ", self.balance, " $")

# Testing the code :
newAccount = BankAccount(2178514584, "Albert", 2700)
# Creating Withdrawal Test
newAccount.Withdrawal(300)
# Create deposit test
newAccount.Deposit(200)
# Display account informations
newAccount.display()
```

```
Account Number : 2178514584
Account Name : Albert
Account Balance : 2600 $
```

```

In [2]: """Circle class
1 - Define a Circle class allowing to create a circleC (0, r)
with center O(a, b) and radius r using the constructor:
    def __init__(self,a,b,r):
        self.a = a
        self.b = b
        self.r = r
2 - Define a Area() method of the class which calculates the area of the circle
3 - Define a Perimeter() method of the class which allows you to
calculate the perimeter of the circle.
4 - Define a testBelongs() method of the class which allows to test whether
A(x, y) belongs to the circle C(0, r) or not."""

from math import pi
class Circle:
    def __init__(self, a, b, r):
        self.a = a
        self.b = b
        self.r = r

    def perimeter (self):
        return 2 * pi * self.r

    def area (self):
        return pi * self.r**2

    # form of the cercle equation
    def formEquation (self, x, y):
        return (x-self.a)**2 + (y-self.b)**2 - self.r**2

    # method to test if given point belong to the circle or not
    def test_belong (self, x, y):
        if (self.formEquation (x, y) == 0):
            print ("the point: (", x, y, ") belongs to the circle C")
        else:
            print ("the point: (", x, y, ") does not belong to the circle C")

# Creating of an instance object
C = Circle (1,2,1)

print ("the perimeter of the circle C is:", C.perimeter() )
print ("the area of circle C is:", C.area())
# we test if the point(1,1) belong to the circle or not
C.test_belong(1,1)
# The output is:
#the perimeter of the circle C is: 6.283185307179586
#the area of circle C is: 3.141592653589793
#the point: ( 1 1 ) belongs to the circle C

```

```

the perimeter of the circle C is: 6.283185307179586
the area of circle C is: 3.141592653589793
the point: ( 1 1 ) belongs to the circle C

```



```

In [3]: """ Computation class:
1 - Create a Coputation class with a default constructor
(without parameters) allowing to perform
various calculations on integers numbers.
2 - Create a method called Factorial() which allows to calculate the factori
integer. Test the method by instantiating the class.
3 - Create a method called Sum() allowing to calculate the sum of the first
integers 1 + 2 + 3 + .. + n. Test this method.
4 - Create a method called testPrim() in the Calculation
class to test the primality of a given integer. Test this method.
4 - Create a method called testPrims() allowing to test if
two numbers are prime between them.
5 - Create a tableMult() method which creates and displays
the multiplication table of a given integer.
Then create an allTablesMult() method to display all the
integer multiplication tables 1, 2, 3, ..., 9.
6 - Create a static listDiv() method that gets all
the divisors of a given integer on new list called Ldiv.
Create another listDivPrim() method that gets all
the prime divisors of a given integer."""

class Computation:
    def __init__(self):
        pass
# --- Factorial -----
    def factorial(self, n):
        j = 1
        for i in range (1, n + 1):
            j = j * i
        return j

# --- Sum of the first n numbers ----
    def sum (self, n):
        j = 1
        for i in range (1, n + 1):
            j = j + i
        return j

# --- Primality test of a number -----
    def testPrim (self, n):
        j = 0
        for i in range (1, n + 1):
            if (n% i == 0):
                j = j + 1
        if (j == 2):
            return True
        else:
            return False

# --- Primality test of two integers -----
    def testprims (self, n, m):

        # initialize the number of commons divisors
        commonDiv = 0
        for i in range (1, n + 1):
            if (n% i == 0 and m% i == 0):
                commonDiv = commonDiv + 1
        if commonDiv == 1:
            print ("The numbers", n, "and", m, "are co-primes")
        else:

```



```

        print ("The numbers", n, "and", m, "are not co-primes")

#---Multiplication table-----
def tableMult (self, k):
    for i in range (1,10):
        print (i, "x", k, "=", i * k)

# --- All multiplication tables of the numbers 1, 2, ..., 9
def allTables (self):
    for k in range (1,10):
        print ("\nthe multiplication table of:", k, "is:")
        for i in range (1,10):
            print (i, "x", k, "=", i * k)

# ----- list of divisors of an integer
def listDiv (self, n):
    # initialization of the list of divisors
    lDiv = []
    for i in range (1, n + 1):
        if (n% i == 0):
            lDiv.append (i)
    return lDiv

# ----- list of prime divisors of an integer -----
def listDivPrim (self, n):
    # initialization of the list of divisors
    lDiv = []
    for i in range (1, n + 1):
        if (n% i == 0 and self.testPrim (i)):
            lDiv.append (i)
    return lDiv

# Instantiation example
Comput= Computation ()
Comput.testprims (13, 7)
print ("List of divisors of 18:", Comput.listDiv (18))
print ("List of prime divisors of 18:", Comput.listDivPrim (18))
Comput.allTables ()

```

The numbers 13 and 7 are co-primes
List of divisors of 18: [1, 2, 3, 6, 9, 18]
List of prime divisors of 18: [2, 3]

the multiplication table of: 1 is:

1 x 1 = 1
2 x 1 = 2
3 x 1 = 3
4 x 1 = 4
5 x 1 = 5
6 x 1 = 6
7 x 1 = 7
8 x 1 = 8
9 x 1 = 9

the multiplication table of: 2 is:

1 x 2 = 2
2 x 2 = 4
3 x 2 = 6
4 x 2 = 8
5 x 2 = 10
6 x 2 = 12
7 x 2 = 14
8 x 2 = 16
9 x 2 = 18

the multiplication table of: 3 is:

1 x 3 = 3
2 x 3 = 6
3 x 3 = 9
4 x 3 = 12
5 x 3 = 15
6 x 3 = 18
7 x 3 = 21
8 x 3 = 24
9 x 3 = 27

the multiplication table of: 4 is:

1 x 4 = 4
2 x 4 = 8
3 x 4 = 12
4 x 4 = 16
5 x 4 = 20
6 x 4 = 24
7 x 4 = 28
8 x 4 = 32
9 x 4 = 36

the multiplication table of: 5 is:

1 x 5 = 5
2 x 5 = 10
3 x 5 = 15
4 x 5 = 20
5 x 5 = 25
6 x 5 = 30
7 x 5 = 35
8 x 5 = 40
9 x 5 = 45

the multiplication table of: 6 is:

1 x 6 = 6

$2 \times 6 = 12$
 $3 \times 6 = 18$
 $4 \times 6 = 24$
 $5 \times 6 = 30$
 $6 \times 6 = 36$
 $7 \times 6 = 42$
 $8 \times 6 = 48$
 $9 \times 6 = 54$

the multiplication table of: 7 is:

$1 \times 7 = 7$
 $2 \times 7 = 14$
 $3 \times 7 = 21$
 $4 \times 7 = 28$
 $5 \times 7 = 35$
 $6 \times 7 = 42$
 $7 \times 7 = 49$
 $8 \times 7 = 56$
 $9 \times 7 = 63$

the multiplication table of: 8 is:

$1 \times 8 = 8$
 $2 \times 8 = 16$
 $3 \times 8 = 24$
 $4 \times 8 = 32$
 $5 \times 8 = 40$
 $6 \times 8 = 48$
 $7 \times 8 = 56$
 $8 \times 8 = 64$
 $9 \times 8 = 72$

the multiplication table of: 9 is:

$1 \times 9 = 9$
 $2 \times 9 = 18$
 $3 \times 9 = 27$
 $4 \times 9 = 36$
 $5 \times 9 = 45$
 $6 \times 9 = 54$
 $7 \times 9 = 63$
 $8 \times 9 = 72$
 $9 \times 9 = 81$

```
In [ ]: """1. Define a Book class with the following attributes: Title,
Author (Full name), Price.
2. Define a constructor used to initialize the attributes of
the method with values entered by the user.
3. Set the View() method to display information for the current book.
4. Write a program to testing the Book class.
Solution
"""

class Book:
    # Question 2
    def __init__(self , Title , Author , Price):
        self.Title    = Title
        self.Author    = Author
        self.Price     = Price

    # Question 3
    def view(self ):
        return ("Book Title: " , self.Title , "Book Author: " ,
                self.Author, "Book Price: " , self.Price)

# Question 4
MyBook = Book("Python Crash Course" , "Eric Matthes" , "23 $")
print( MyBook.view())
# The output: ('Book Title: ', 'Python Crash Course', 'Book Author: ',
'Eric Matthes', 'Book Price: ', '23 $')
```

```

In [1]: class flashcard:
        def __init__(self, word, meaning):
            self.word = word
            self.meaning = meaning
        def __str__(self):

            #we will return a string
            return self.word+' ( '+self.meaning+' )'

flash = []
print("welcome to flashcard application")

#the following loop will be repeated until
#user stops to add the flashcards
while(True):
    word = input("enter the name you want to add to flashcard : ")
    meaning = input("enter the meaning of the word : ")

    flash.append(flashcard(word, meaning))
    option = int(input("enter 0 , if you want to add another flashcard : "))

    if(option):
        break

# printing all the flashcards
print("\nYour flashcards")
for i in flash:
    print(">", i)

```

```

welcome to flashcard application
enter the name you want to add to flashcard : fcsp
enter the meaning of the word : fundamental of computer science python
enter 0 , if you want to add another flashcard : 1

```

```

Your flashcards
> fcsp ( fundamental of computer science python )

```



```

In [5]: # Create class Student
        """Write a program to build a simple Student Management System using Python
        perform the following operations:
        Accept
        Display
        Search
        Delete
        Update
        Accept - This method takes details from the user like name, roll number,
        and marks for two different subjects.
        Display - This method displays the details of every student.
        Search - This method searches for a particular student from the list of
        students. This method will ask the user for roll number and then search
        according to the roll number
        Delete - This method deletes the record of a particular student with a
        matching roll number.
        Update - This method updates the roll number of the student. This method
        will ask for the old roll number and new roll number. It will replace the
        old roll number with a new roll number."""
        class Student:
            def __init__(self, name, roll, s1, s2):
                self.name = name
                self.roll = roll
                self.s1 = s1
                self.s2 = s2
            # Function to create and append students
            def accept(self, Name, Roll, score1, score2):
                obj = Student(Name, Roll, score1, score2)
                ls.append(obj)
            # Display student details
            def display(self, obj):
                print("Name : ", obj.name)
                print("RollNo : ", obj.roll)
                print("Score1 : ", obj.s1)
                print("Score2 : ", obj.s2)
                print("\n")
            # Search Function
            def search(self, rn):
                for i in range(ls.__len__()):
                    if (ls[i].roll == rn):
                        return i
            # Deletion
            def delete(self, rn):
                i = obj1.search(rn)
                del ls[i]
            # Update Function
            def update(self, rn, No):
                i = obj1.search(rn)
                rolln = No
                ls[i].roll = rolln;
        ls = []
        # Object of class
        obj1 = Student('', 0, 0, 0)
        print("\nOperations used, ")
        print("\n1.Accept Student details\n"
              "2.Display Student Details\n"
              "3.Search Details of a Student\n"
              "4.Delete Details of Student"
              "\n5.Update Student Details\n6.Exit")
        obj1.accept("A", 1, 100, 100)
        obj1.accept("B", 2, 90, 90)

```

```
obj1.accept("C", 3, 80, 80)
print("\n")
print("\nList of Students\n")
for i in range(ls.__len__()):
    obj1.display(ls[i])
print("\n Student Found, ")
s = obj1.search(2)
obj1.display(ls[s])
obj1.delete(2)
print(ls.__len__())
print("List after deletion")
for i in range(ls.__len__()):
    obj1.display(ls[i])
obj1.update(3, 2)
print(ls.__len__())
print("List after updation")
for i in range(ls.__len__()):
    obj1.display(ls[i])
print("Thank You !")
```


Operations used,

- 1.Accept Student details
- 2.Display Student Details
- 3.Search Details of a Student
- 4.Delete Details of Student
- 5.Update Student Details
- 6.Exit

List of Students

Name : A
RollNo : 1
Score1 : 100
Score2 : 100

Name : B
RollNo : 2
Score1 : 90
Score2 : 90

Name : C
RollNo : 3
Score1 : 80
Score2 : 80

Student Found,
Name : B
RollNo : 2
Score1 : 90
Score2 : 90

2
List after deletion
Name : A
RollNo : 1
Score1 : 100
Score2 : 100

Name : C
RollNo : 3
Score1 : 80
Score2 : 80

2
List after updation
Name : A
RollNo : 1
Score1 : 100
Score2 : 100

Name : C
RollNo : 2
Score1 : 80
Score2 : 80

Thank You !

```
In [1]: import math

class Point:
    """Class to handle point in a 2 dimensions space"""

    def __init__(self, x, y):
        """
        :param x: the value on the X-axis
        :type x: float
        :param y: the value on the Y-axis
        :type y: float
        """
        self.x = x
        self.y = y

    def show(self):
        """
        :return: the coordinate of this point
        :rtype: a tuple of 2 elements (float, float)
        """
        return self.x, self.y

    def move(self, x, y):
        """
        :param x: the value to move on the X-axis
        :type x: float
        :param y: the value to move on the Y-axis
        :type y: float
        """
        self.x += x
        self.y += y

    def dist(self, pt):
        """
        :param pt: the point to compute the distance with
        :type pt: :class:`Point` object
        :return: the distance between this point and pt
        :rtype: int
        """
        dx = pt.x - self.x
        dy = pt.y - self.y
        return math.sqrt(dx ** 2 + dy ** 2)
```

```
In [3]: p1=Point(5,6)
print(p1.show())
p1.move(7,8)
print(p1.show())
x=Point(7,5)
p1.dist(x)
```

```
(5, 6)
(12, 14)
```

```
Out[3]: 10.295630140987
```

```
In [4]: """Stacks and Queues. Write a class which defines a data structure that can
There are four methods that should be implemented:
shift() returns the first element and removes it from the list
unshift() "pushes" a new element to the front or head of the list
push() adds a new element to the end of a list
pop() returns the last element and removes it from the list
GENERATE CUSTOM EXCEPTION for shift() and pop() method if there is no element
```

```
class StackQueue:
    def __init__(self, L):
        self.L = L
    def shift(self):
        if len(self.L) == 0:
            pass
            raise Exception('List is empty')
        try:
            x = self.L.pop(0)
            return x
        except Exception:
            print(Exception)
    def unshift(self, n):
        self.L.insert(0, n)
    def push(self, n):
        self.L.append(n)
    def pop(self):
        if len(self.L) == 0:
            raise Exception('List is empty')
        try:
            x = self.L.pop()
            return x
        except Exception:
            print(Exception)
    def display(self):
        return self.L
```

```
sq = StackQueue([1,4,6,8,9])
print(sq.shift())
sq.unshift(7)
sq.push(10)
print(sq.pop())
print(sq.display())
```

```
1
10
[7, 4, 6, 8, 9]
```



```

In [7]: class Pizza:
    def __init__(self, size, toppings, cheese):
        self.size = size
        self.toppings = toppings
        self.cheese = cheese

    def price(self):
        self.cost = 0
        if self.size == 'small':
            self.cost += 50
        elif self.size == 'medium':
            self.cost += 100
        else:
            self.cost += 200
        topping_prices_20 = ['corn', 'tomato', 'onion', 'capsicum']
        topping_prices_50 = ['mushroom', 'olives', 'broccoli']
        for topping in self.toppings:
            if topping in topping_prices_20:
                self.cost += 20
            else:
                self.cost += 50
        #for cheese
        self.cost += 50 * len(self.cheese)
        return self.cost

class Order:
    def __init__(self, name, customerid):
        self.name = name
        self.customerid = customerid

    def order(self, n):
        self.pizzas = []
        for i in range(n):
            toppings = []
            cheese = []
            print('Customize Pizza', i+1)
            size = input('Select size: ')
            t = int(input('How many toppings: '))
            for i in range(t):
                toppings.append(input('Enter toppings: '))
            t = int(input('How many cheese: '))
            for i in range(t):
                cheese.append(input('Enter cheese: '))
            self.pizzas.append(Pizza(size, toppings, cheese))

    def bill(self):
        self.total = 0
        count = 1
        for p in self.pizzas:
            print('Pizza', count)
            print(p.size, p.toppings, p.cheese)
            self.total += p.price()
            count += 1
        print('Total bill amount:', self.total)

number=int(input("How many pizzas you want to order: "))
order1 = Order('Kavit', 1)
order1.order(number)

```

```
order1.bill()
```

How many pizzas you want to order: 2

Customize Pizza 1

Select size: small

How many toppings: 2

Enter toppings: tomato

Enter toppings: onion

How many cheese: 1

Enter cheese: feta

Customize Pizza 2

Select size: large

How many toppings: 1

Enter toppings: olives

How many cheese: 1

Enter cheese: feta

Pizza 1

small ['tomato', 'onion'] ['feta']

Pizza 2

large ['olives'] ['feta']

Total bill amount: 440

```
In [8]: """Write a class called Product. The class should have fields
called name, amount, and price, holding the product's name, the number
of items of that product in stock, and the regular price of the product.
There should be a method get_price that receives the number of items to be
bought and returns a the cost of buying that many items, where the regular
price is charged for orders of less than 10 items, a 10% discount is applied
orders of between 10 and 99 items, and a 20% discount is applied for orders
100 or more items. There should also be a method called
make_purchase that receives the number of items to be bought and decreases
amount by that much."""

class Product:

    def __init__(self, name, amount, price):
        self.name = name
        self.amount = amount
        self.price = price

    def get_price(self, number_to_be_bought):
        discount = 0
        if number_to_be_bought < 10:
            pass
        elif 10 <= number_to_be_bought < 99:
            discount = 10
        else:
            discount = 20
        price = (100 - discount) / 100 * self.price
        return price * number_to_be_bought

    def make_purchase(self, quantity):
        self.amount -= quantity

# name = input('name:')
# amount = int(input('Digit amount of items'))
# price = int(input('Digit price of items'))

name, amount, price = 'shoes', 200, 33

shoes = Product(name, amount, price)
# quantity = int(input('Digit amount of items to buy'))

q1 = 4
print(f'cost for {q1} {shoes.name} = {shoes.get_price(q1)}')
shoes.make_purchase(q1)
print(f'remaining stock: {shoes.amount}\n')

q2 = 12
print(f'cost for {q2} {shoes.name} = {shoes.get_price(q2)}')
shoes.make_purchase(q2)
print(f'remaining stock: {shoes.amount}\n')

q3 = 112
print(f'cost for {q3} {shoes.name} = {shoes.get_price(q3)}')
shoes.make_purchase(q3)
print(f'remaining stock: {shoes.amount}\n')
```

cost for 4 shoes = 132.0
remaining stock: 196

cost for 12 shoes = 356.4
remaining stock: 184

cost for 112 shoes = 2956.8
remaining stock: 72


```
In [9]: """You need to create the foundations of an
e-commerce engine for a B2C (business-to-consumer) retailer.
You need to have a class for a customer called User, a class for
items in inventory called Item, and a shopping cart class called Cart.
Items go in Carts, and Users can have multiple Carts. Also, multiple items c
including more than one of any single item."""

class User:
    def __init__(self, id, name):
        self.id = id
        self.name = name
    def display_user(self):
        print('ID', self.id, 'Name:', self.name)

class Item:
    def __init__(self, id, name, price, sold, available):
        self.id = id
        self.name = name
        self.price = price
        self.sold = sold
        self.available = available

class Cart:
    def __init__(self, user):
        self.user = user
        self.cart_items = []
    def insert_items(self, item, quantity):
        for i in range(quantity):
            if item.available == 0:
                print('Out of stock')
                break
            self.cart_items.append(item)
            item.sold += 1
            item.available -= 1
    def display_cart(self):
        print('This Cart belongs to', self.user.name, 'with ID', self.user.i
        self.total = 0
        for i in self.cart_items:
            print('Item', i.name)
            self.total += i.price
        print('Total price = ', self.total)

#Creating a user
user1 = User(1, 'Kavit')
user1.display_user()

#Creating items
apple = Item(1, 'apple', 100, 0, 10)
greencoconut = Item(2, 'green coconut', 150, 0, 1)
milk = Item(2, 'milk', 24, 0, 100)

#Creating two carts for user1
cart1 = Cart(user1)
cart2 = Cart(user1)

#Adding items to cart1
cart1.insert_items(apple, 2)
cart1.insert_items(milk, 3)
```

[illegible]

```
In [ ]: """Create python class Atm.
create menu() to give menu options to user.
create create_pin(),change_pin(),check_bal(),wthdraw(),deposit() methods.
output:
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
1
enter your pin2345
enter balance10000
pin created successfully
"""
```


In [20]:

```
class Atm:
    # constructor
    def __init__(self):
        # print(id(self))
        self.pin = ''
        self.balance = 0
        self.menu()
    def menu(self):
        user_input = input("""
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
""")
        if user_input == '1':
            self.create_pin()
        elif user_input == '2':
            self.change_pin()
        elif user_input == '3':
            self.check_balance()
        elif user_input == '4':
            self.withdraw()
        else:
            exit()
    def create_pin(self):
        user_pin = input('enter your pin')
        self.pin = user_pin
        user_balance = int(input('enter balance'))
        self.balance = user_balance
        print('pin created successfully')
        self.menu()
    def change_pin(self):
        old_pin = input('enter old pin')
        if old_pin == self.pin:
            # Let him change the pin
            new_pin = input('enter new pin')
            self.pin = new_pin
            print('pin change successful')
        else:
            print('enter correct pin')
            self.menu()
    def check_balance(self):
        user_pin = input('enter your pin')
        if user_pin == self.pin:
            print('your balance is ', self.balance)
        else:
            print('enter correct pin')
            self.menu()
    def withdraw(self):
        user_pin = input('enter the pin')
        if user_pin == self.pin:
            # allow to withdraw
            amount = int(input('enter the amount'))
            if amount <= self.balance:
                self.balance = self.balance - amount
                print('withdrawl successful.balance is ', self.balance)
            else:
```

```
                print('increase your balance')
            else:
                print('enter correct pin')
            self.menu()
sbi=Atm()
```

```
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
1
enter your pin2345
enter balance10000
pin created successfully
```

```
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
2
enter old pin2345
enter new pin5678
pin change successful
```

```
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
3
enter your pin5678
your balance is 10000
```

```
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
4
enter the pin5678
enter the amount2345
withdrawl successful.balance is 7655
```

```
Hi how can I help you?
1. Press 1 to create pin
2. Press 2 to change pin
3. Press 3 to check balance
4. Press 4 to withdraw
5. Anything else to exit
6
```

